

Programming paradigm

- an approach to solve problems using some programming language
- or also we can say it is a method to solve a problem using tools and techniques that are available to us following some approach.
- There are lots of programming languages that are known but all of them need to follow some strategy when they are implemented and this methodology/strategy is paradigms.
- **A programming paradigm is a set of programming concepts and styles that define how to write and organize code. It's a guiding philosophy for writing code,** and it helps developers think about and structure their code in a unique way.

Programming Paradigms

Imperitive Programming Paradigm

Procedural programming Paradigm

Object Oriented Programming

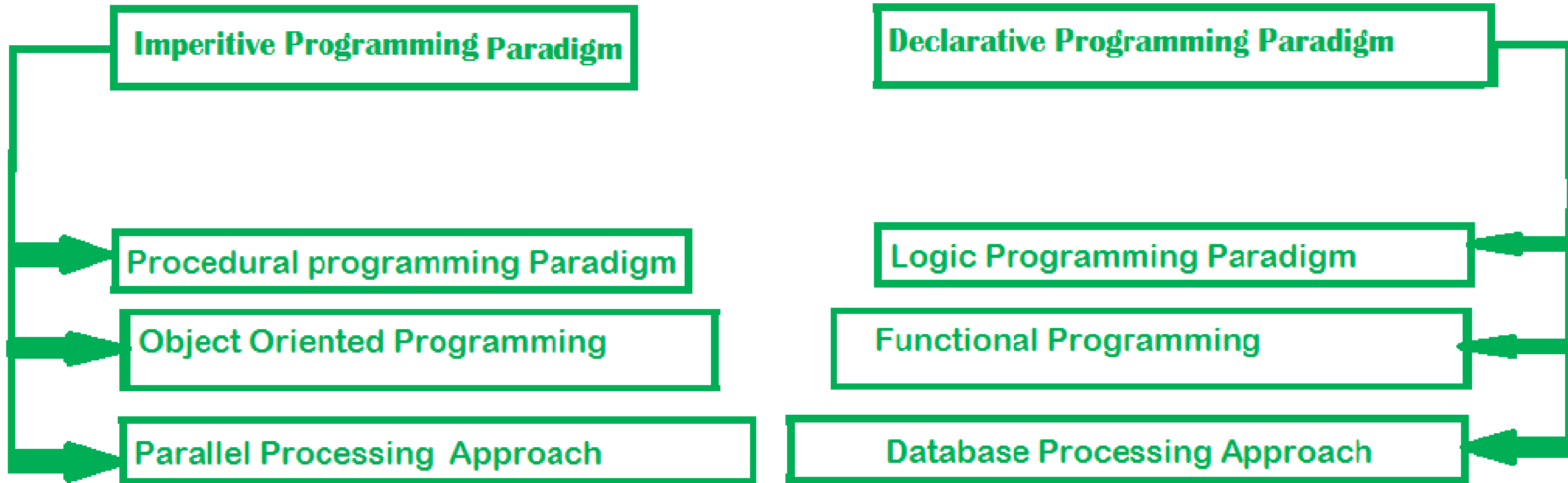
Parallel Processing Approach

Declarative Programming Paradigm

Logic Programming Paradigm

Functional Programming

Database Processing Approach



- **Programming paradigms are not languages or tools. You can't "build" anything with a paradigm.**
- **They're more like a set of ideals and guidelines that many people have agreed on, followed, and expanded upon.**
- **there are also "multi-paradigm" languages, meaning you can adapt your code to fit a certain paradigm or another (JavaScript and Python are good examples).**

Imperative Programming

- Imperative programming consists of sets of detailed instructions that are given to the computer to execute in a given order. It's called "imperative" because as programmers we dictate exactly what the computer has to do, in a very specific way.
- Imperative programming focuses on describing how a program operates, step by step.
- Say you want to bake a cake. Your imperative program to do this might look like this:
 - 1- Pour flour in a bowl
 - 2- Pour a couple eggs in the same bowl
 - 3- Pour some milk in the same bowl
 - 4- Mix the ingredients
 - 5- Pour the mix in a mold
 - 6- Cook for 35 minutes
 - 7- Let chill
 -

Procedural Programming

- Procedural programming is a derivation of imperative programming, adding to it the feature of functions (also known as "procedures" or "subroutines").
- In procedural programming, the user is encouraged to subdivide the program execution into functions, as a way of improving modularity and organization.
- Following our cake example, procedural programming may look like this:

function pourIngredients() {

- Pour flour in a bowl
- Pour a couple eggs in the same bowl
- Pour some milk in the same bowl

}

function mixAndTransferToMold() {

- Mix the ingredients
- Pour the mix in a mold

}

- Procedural programming is a programming paradigm built around the idea that programs are sequences of instructions to be executed.
- They focus heavily on splitting up programs into named sets of instructions called procedures, analogous to functions.
- The programming language C is an example of a language that is purely procedural.
- However, many programming languages support multiple paradigms. For example Python, which is a popular general-purpose programming language, allows you to code using procedural, object-oriented, or functional techniques.

Main Characteristics of Procedural Programming

- Procedural programming is distinct from other programming paradigms in three ways: it uses loops instead of recursion, and its code allows for data mutation.
- Use of loops: Loops, such as for loops, while loops, or do-while loops, are commonly used to repeat a set of instructions until a specific condition is met. Loops are essential in procedural programming.
- Absence (or reduced presence) of recursion: While recursion is entirely possible in procedural programming, it is used less frequently than in other types of programming, such as functional programming. Loops are typically preferred over recursion in procedural programming.
- Emphasis on data mutation: In procedural programming, data is often mutable, meaning it can be modified, allowing the programmer to process the data and update variables as needed by the program.

Benefits

- Splitting code into smaller chunks has many benefits:
- It is much easier to test and debug, e.g. tracing 20 lines of code instead of 200 lines of code.
- Subroutines can be called many times, reducing the amount of repeated code.
- Subroutines can manipulate shared data.
- Subroutines can be saved in libraries and imported into programs when they are needed. This means that code is reusable; it's good if you can write code once and make use of it many times.
- Very large programs can be worked on by a whole team of programmers who can develop subroutines concurrently (at the same time) so that a project can be completed faster.

Benefits

- The code is reusable: One of the primary benefits of procedural programming is the ability to reuse code in different parts of the program without needing to rewrite it. Once a procedure is written, it can be called as many times as needed, resulting in simpler and more readable code.
- The code is clearer: Breaking down a program into procedures makes the code clearer and easier to understand. Each procedure has a specific role, assisting developers in following their program more easily and effectively.
- Code errors are more easily corrected: By isolating specific tasks in procedures, developers can test and debug each part of the program independently. In case of an error in a program organized with procedures, a developer can locate and correct the error more quickly than in poorly organized code, leading to significant time and efficiency savings.

Drawbacks

- One criticism of the procedural paradigm is that it focuses on what needs to be done, rather than on the integrity of the data that it manipulates.
- Another factor that can cause issues is that in the procedural paradigm it is possible to use global variables. The problem with global variables is that, since they can be accessed and modified by every subroutine in a program, it is really hard to work out where a value is changed. If you are trying to debug a program, you may have to consider every subroutine that modifies the global state. That is possible, but with a really large application, it is very difficult to do.
- Of course, the use of global variables is not a necessary component of the procedural paradigm, but it is allowed. Global variables should always be used very carefully. Indeed, there is nothing in the procedural paradigm that truly protects data. Data is passed into subroutines, used, and sometimes modified, and the programmer must be fully aware of what they should and shouldn't do, and abide by the rules of good practice to ensure data integrity.

Functional Programming Paradigm

- programming paradigm in which we try to bind everything in pure mathematical functions.
- It is a declarative style. Its main focus is on “what to solve,” in contrast to an imperative style, where the main focus is on “how to solve.”
- It uses expressions instead of statements. An expression is evaluated to produce a value, whereas a statement is executed to assign variables.
- Programming Languages that support functional programming: Haskell, JavaScript, Python, Scala, Erlang, Lisp, ML, Clojure, OCaml, Common Lisp, Racket.

Concepts of Functional Programming

Pure functions

Recursion

Referential transparency

Functions are First-Class and can be Higher-Order

Variables are Immutable

Pure Functions

- These functions have two main properties. First, they always produce the same output for same arguments irrespective of anything else.
- Secondly, they have no side-effects i.e. they do not modify any arguments or local/global variables or input/output streams.
- Later property is called immutability. The pure function's only result is the value it returns. They are deterministic.
- Programs done using functional programming are easy to debug because pure functions have no side effects or hidden I/O. Pure functions also make it easier to write parallel/concurrent applications. When the code is written in this style, a smart compiler can do many things – it can parallelize the instructions, wait to evaluate results when needing them, and memorize the results since the results never change as long as the input doesn't change.

Example of the Pure Function

```
sum(x, y)    // sum is function taking x and y as arguments  
return x + y // sum is returning sum of x and y without changing them
```

Recursion

- **There are no “for” or “while” loop in functional languages. Iteration in functional languages is implemented through recursion.**
- **Recursive functions repeatedly call themselves, until it reaches the base case.**

Variables are Immutable

In functional programming, we can't modify a variable after it's been initialized. We can create new variables – but we can't modify existing variables, and this really helps to maintain state throughout the runtime of a program. Once we create a variable and set its value, we can have full confidence knowing that the value of that variable will never change.

Referential Transparency

Referential transparency is a property of a function that allows it to be replaced by its equivalent output. In simpler terms, if you call the function a second time with the same arguments, you're guaranteed to get the same returning value.

Example

Let's consider the following function, which computes the sum of 2 numbers:

```
algorithm add(a, b):
```

```
  // This function is referentially opaque.
```

```
  int a, b
```

```
  read a, b
```

```
  return a + b
```

The function is referentially opaque because the parameters defined in its signature aren't all of the resources the function uses.

- The a and b variables are read from standard input inside the function body, so two function calls with the same arguments may produce different results.
- In order to make the function referentially transparent, we need to extract the input part from the function:

```
algorithm add(a, b):  
    return a + b
```

```
algorithm main():
```

```
    // This function handles user input and calls the add function to compute the sum of two  
    numbers.  
    int a, b  
    read a, b  
    int s = add(a, b)  
    print s  
    return 0Copy
```

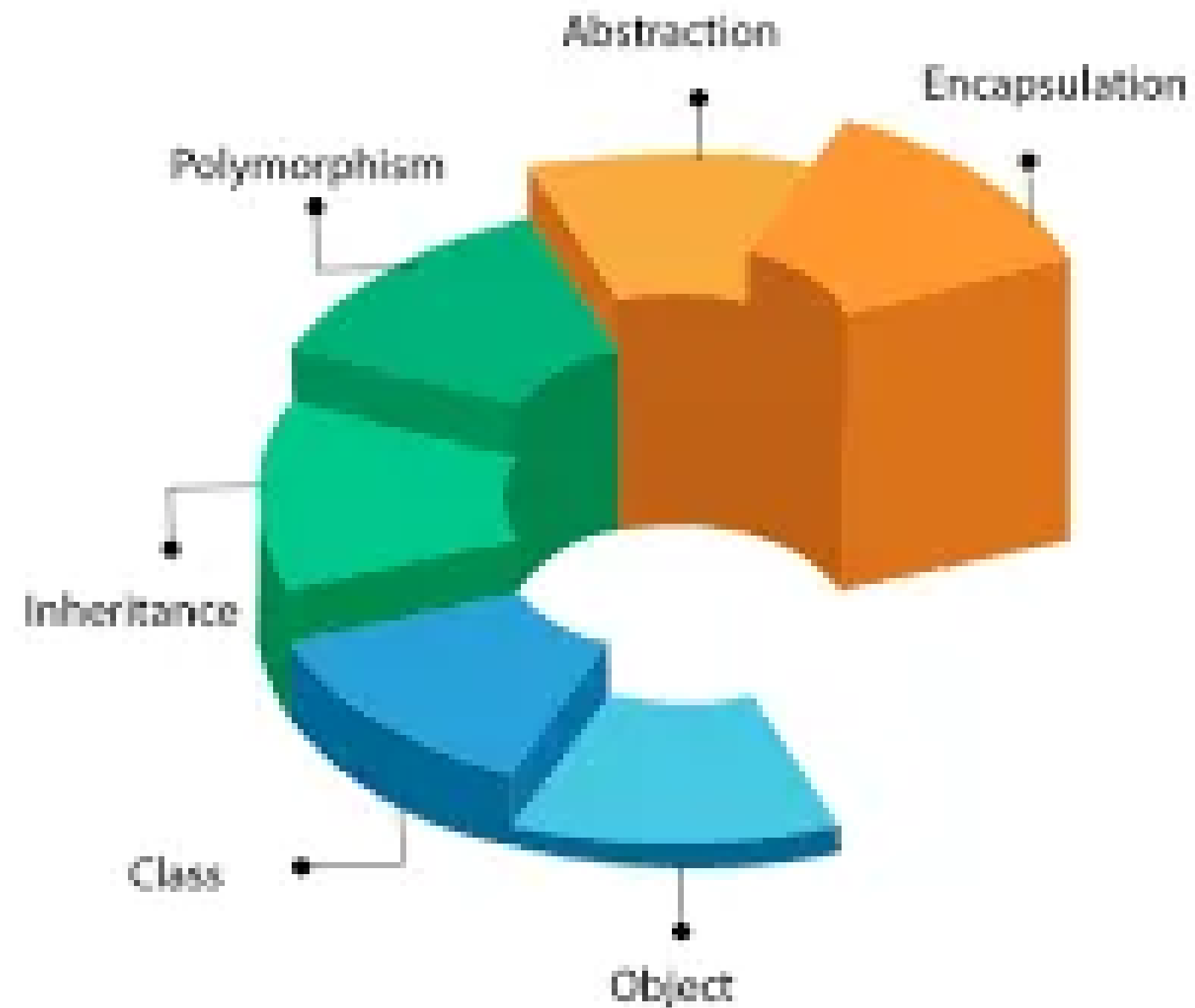
- **Note that this doesn't prevent us from interacting with user input, we're still able to read the values for a and b. Thus, in the code snippet, the add function is transparent, while the main function isn't.**

Advantages of Functional Programming

- Pure functions are easier to understand because they don't change any states and depend only on the input given to them. Whatever output they produce is the return value they give. Their function signature gives all the information about them i.e. their return type and their arguments.
- The ability of functional programming languages to treat functions as values and pass them to functions as parameters make the code more readable and easily understandable.
- Testing and debugging is easier. Since pure functions take only arguments and produce output, they don't produce any changes don't take input or produce some hidden output. They use immutable values, so it becomes easier to check some problems in programs written uses pure functions.
- It is used to implement concurrency/parallelism because pure functions don't change variables or any other data outside of it.

Object oriented programming paradigm

OOPs (Object-Oriented Programming System)

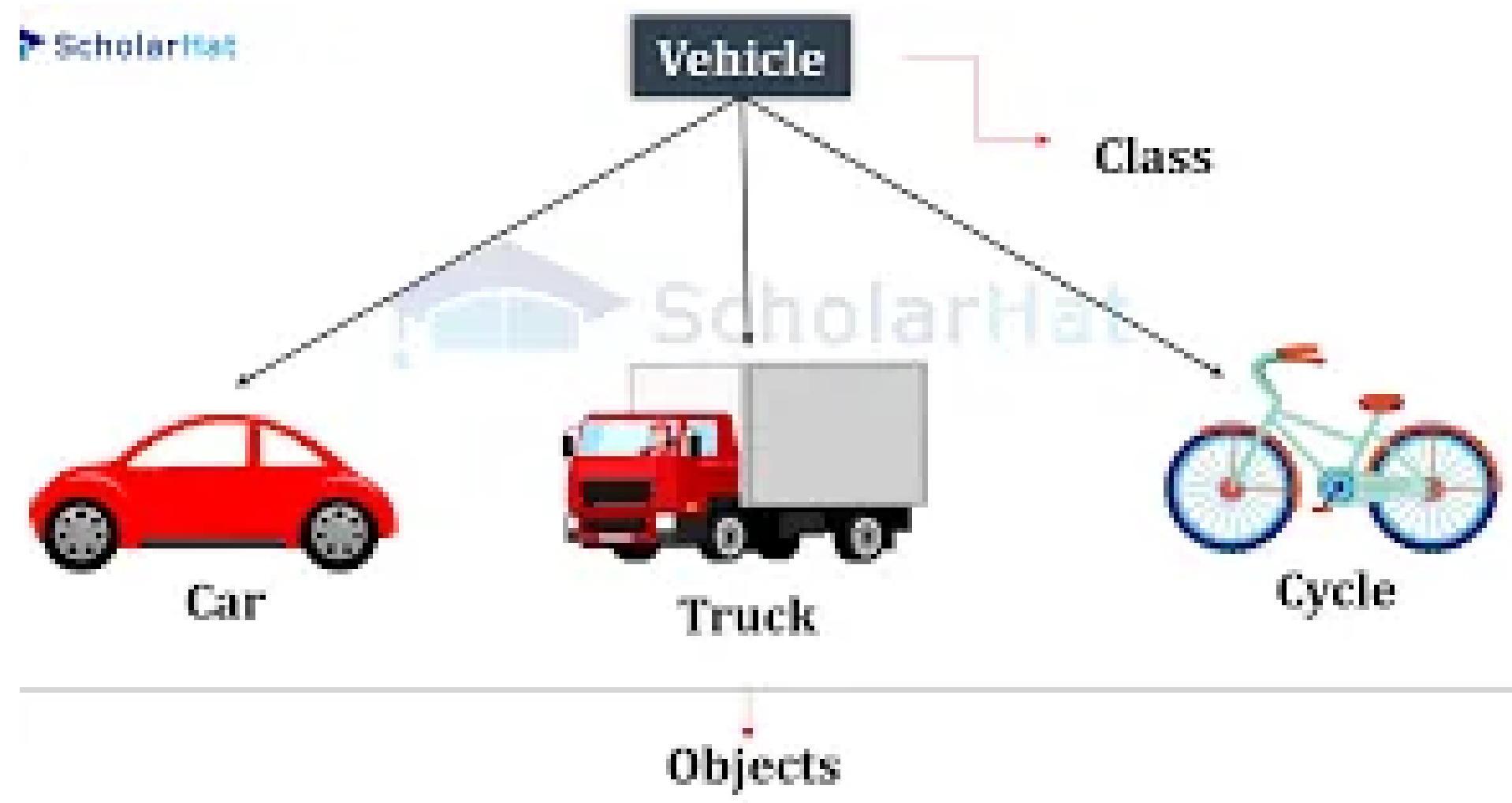


- OOP is a programming paradigm used for software development. Its basis is modeling objects and relationships in real world. OOP paradigm of our software projects; It makes it more modular, readable, understandable and easy to maintain

The OOP paradigm has 5 basic pillars:

1- Class and object relationships:

- Classes are design templates that contain the attributes and methods (behavior) of objects. Objects are instances of these classes.
- By creating an object of a class, we can access and implement the attributes and methods of that class. Classes and objects make our code more modular and make it easier to share similar features and behaviors.

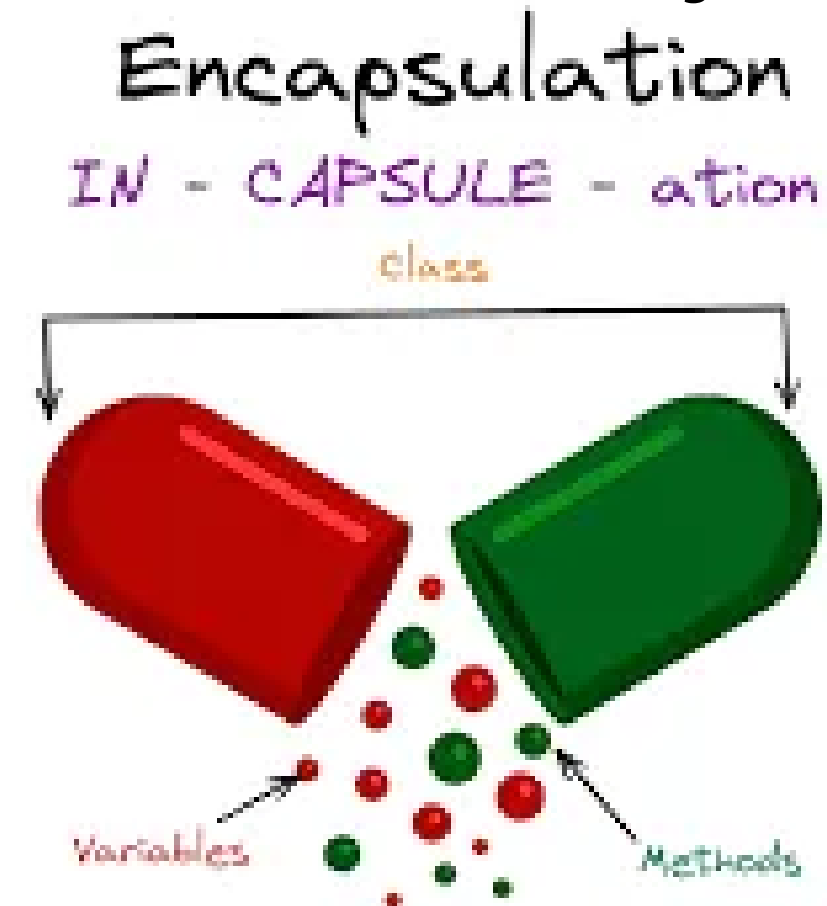


Benefits of classes and objects:

- 1 – Classes break code into smaller manageable pieces. Thus, while each class does its own job, the code can be understood more easily and maintained more easily.
- 2 – Development speed increases as it reduces code repetition.
- 3 – Prepares the basis for other OOP principles.

2- Encapsulation:

The concept of Encapsulation is used to introduce access control to the object's data. Access is provided using certain modifiers. Anyone cannot access the data as they wish.



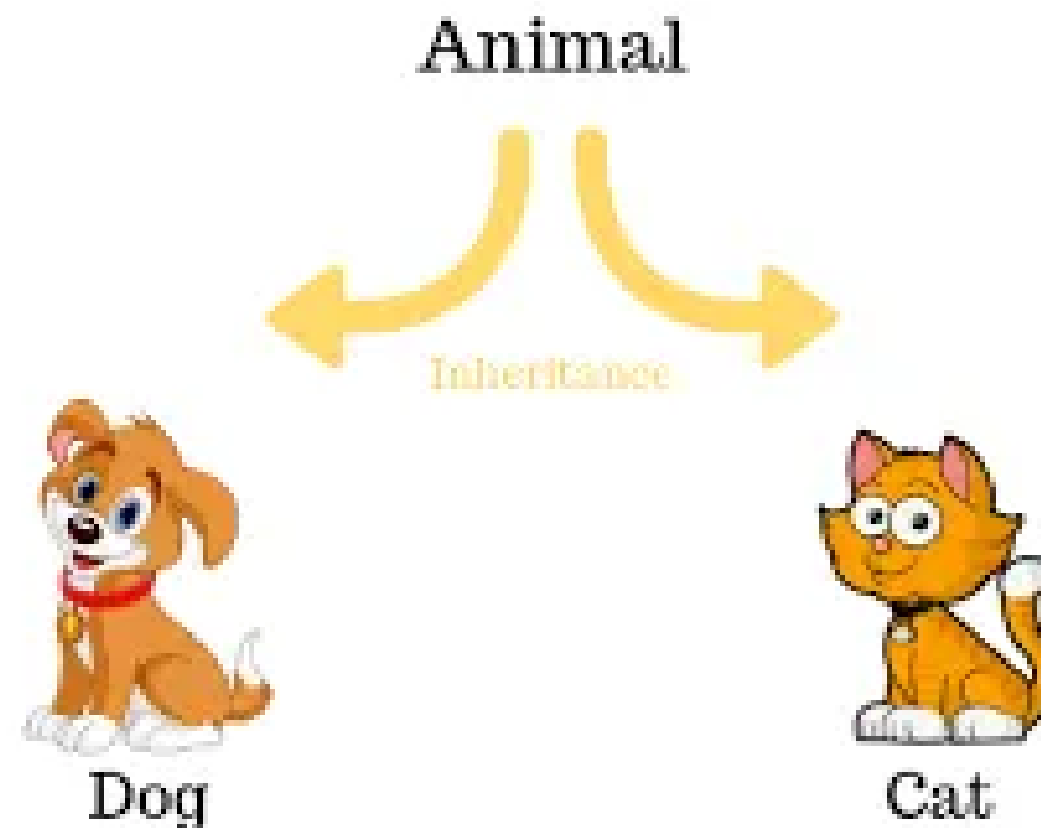
3- Abstraction:

- Abstraction is one of the most basic and important concepts of OOP and software engineering.
- Abstraction is the principle of hiding some details in order to emphasize important features in complex systems.
- In this way, it becomes easier to see the big picture of the project and the operation becomes simpler and more understandable.
- **Benefits of Abstarction:**
- 1 — Reducing complexity: A system with high complexity can be represented using a high-level abstraction.
- 2 — Reusability: Abstraction of base classes or data types increases the reusability of the code. This speeds up the implementation process and reduces code repetition.
- 3 — Hiding details: Unnecessary details are abstracted to focus on what is what. In other words, systems, objects, classes exist as far as we can see.

- Example of Abstraction: Database objects represent data stored in the database. We can access data using the functions offered by the database object, but we have no knowledge of how it is stored and managed.

4- Inheritance:

Inheritance is defined as a class inheriting features and behaviors from another base class. Inheritance represents the parent-child relationship between classes.

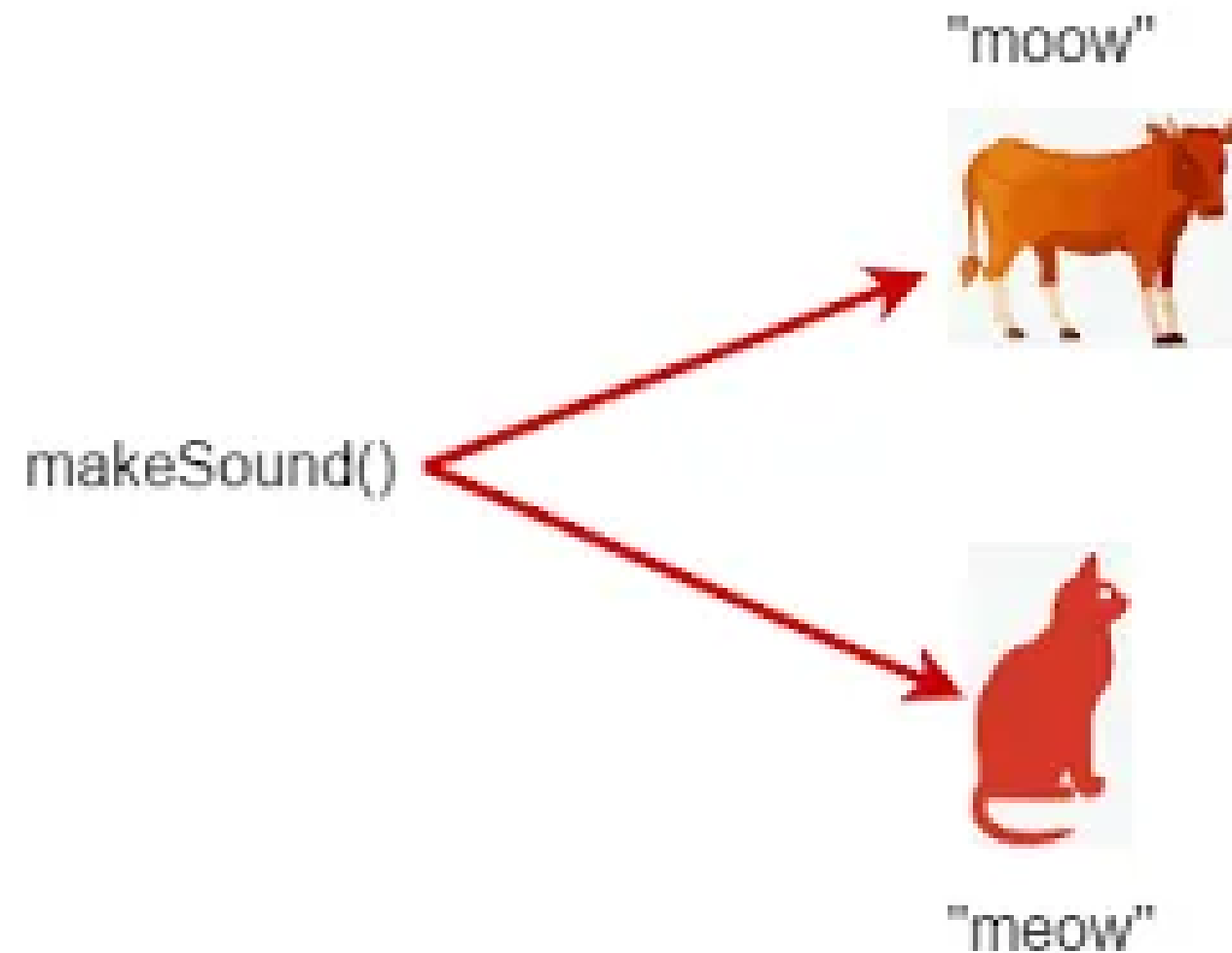


Benefits of inheritance:

- 1 — Increases code reusability and prevents code duplication.
- 2 — We create general classes by bringing similar classes together through inheritance. In this way, it offers a high-level and abstract perspective.
- 3 — Derived classes can extend the features of base classes and add features and behaviors. This allows the code to be more flexible and customizable.
- 4 — Thanks to the hierarchical structure of inheritance, complex systems become more orderly.

5- Polymorphism:

- First of all, in order to talk about polymorphism, it is necessary to consider the concept of “override”. Override is when the subclass changes a certain behavior or function of the parent class according to its own needs. Polymorphism also supports and maintains this. It redesigns the content of a method inherited from the superclass according to its own needs and logic.



Benefits of polymorphism:

- 1 — Enables objects in the same class hierarchy to behave in different ways using the same interface.**
- 2 — Provides flexibility by customizing different objects.**
- 3 — The reusability of the code increases.**

Differences between Procedural and Object Oriented Programming

- **Procedural Programming**

- Procedural Programming can be defined as a programming model which is derived from structured programming, based upon the concept of calling procedure. Procedures, also known as routines, subroutines or functions, simply consist of a series of computational steps to be carried out. During a program's execution, any given procedure might be called at any point, including by other procedures or itself.
- Languages used in Procedural Programming: FORTRAN, ALGOL, COBOL, BASIC, Pascal and C.

Object-Oriented Programming

- Object-oriented programming can be defined as a programming model which is based upon the concept of objects. Objects contain data in the form of attributes and code in the form of methods. In object-oriented programming, computer programs are designed using the concept of objects that interact with the real world.
- Java, C++, C#, Python,
- PHP, JavaScript, Ruby, Perl,
- Objective-C, Dart, Swift, Scala.
-

Procedural Oriented Programming	Object-Oriented Programming
In procedural programming, the program is divided into small parts called functions.	In object-oriented programming, the program is divided into small parts called objects.
Procedural programming follows a top-down approach.	Object-oriented programming follows a bottom-up approach.
There is no access specifier in procedural programming.	Object-oriented programming has access specifiers like private, public, protected, etc.
Adding new data and functions is not easy.	Adding new data and function is easy.
Procedural programming does not have any proper way of hiding data so it is less secure.	Object-oriented programming provides data hiding so it is more secure.
In procedural programming, overloading is not possible.	Overloading is possible in object-oriented programming.

In procedural programming, there is no concept of data hiding and inheritance.	In object-oriented programming, the concept of data hiding and inheritance is used.
In procedural programming, the function is more important than the data.	In object-oriented programming, data is more important than function.
Procedural programming is based on the unreal world.	Object-oriented programming is based on the real world.
Procedural programming is used for designing medium-sized programs.	Object-oriented programming is used for designing large and complex programs.
Procedural programming uses the concept of procedure abstraction.	Object-oriented programming uses the concept of data abstraction.
Code reusability absent in procedural programming,	Code reusability present in object-oriented programming.
Examples: C, FORTRAN, Pascal, Basic, etc.	Examples: C++, Java, Python, C#, etc.

Logic Programming

- Logical programming is a paradigm in which logical circuits are used to represent the problems with the help of facts and rules. Knowledge is represented with the help of logic. This knowledge can be modified with the help of inference. This paradigm helps in knowing the ways to accomplish certain goals.
- Logical language is based on another paradigm where sentences are evaluated by rules based on pure logic.
- A Logical language has features that allow the programmer to write statements expressing that, if some set of facts are true, then another set of facts are true.
- **this is the trend that languages are heading towards. Java and Python started as object oriented, but have had features from functional programming (lambdas, map, filter, reduce) for a while now. Prolog is mixed imperative and logical. Scala was designed to be mixed paradigm from day one.**

Functional Programming	Logical Programming
It uses one or more functions in a program.	It uses predicates which do not have a return value.
Functions have to be written and executed in this type of programming.	Statements are used in this type of programming. These statements represent the problems in the form of logic.
Functional programming is performed to handle symbolic computation.	Logical programming is performed to diagnose faults, machine learning, etc.
Functional programming depends on functions which can be user-defined or available in the libraries of a programming language.	Logical programming is based on logic.
The languages that use functional programming are Clojure, OCaml, Wolfram language, etc.	The programming languages that use logic programming are Absys, Cycl, Alice, etc.
Code redundancy is reduced and it can also be used to solve complex problems.	It is a data-driven programming which is used to provide knowledge.
Testing of a functional program is easy.	Testing is difficult comparatively.

